

The Mathematics of Visibility

What makes a mark visible on a screen — and why it is not enough for two colours to be “different”. The WCAG formula, the measured palette of vig yazokrad.hu, and a live colour-vision-deficiency simulator.

From the workshop of an emergency-call app · 19 September 2026

This page grew out of one concrete defect that lived inside an **emergency-call application**. Every number on it is **measured**, not estimated — and at the end you will find the questions we do not know the answers to.

A PERSONAL REQUEST

My name is Péter Kürtösi, and I am asking for your help. I am an engineer, not a colour scientist. I build this system for elderly and visually impaired people, where an invisible mark is not a question of taste — there, somebody fails to find help.

The defect described below was *not* found by me. What I have assembled here is the mathematics I can program against. If I have misunderstood any of it, if there is a better path, or if you can answer the four questions in section 7 — **please tell me**. Any remark you make is worth more than the effort it costs you.

Any feedback at all is welcome — a correction, a disagreement, a reference, or simply “this is wrong”.

Péter Kürtösi · peter@vig yazokrad.hu

1. The defect that started it

The app’s home screen is a grid of tiles, one person per tile. A small icon at the bottom of each tile told you whether tapping it would place a *call* or send a *message*. The icon’s colour was fine — on tiles with a photograph it was perfectly legible.

Then someone who merely glanced at the phone said this:

“*The envelope is exactly the same green as the tile behind it. Nobody can see it.*”

The name band under an illustrated avatar is itself a pastel. The very same icon that was perfectly visible over a photograph simply **disappeared** there. And when we measured it, the situation turned out to be worse than the eye suggested. The icon colour was #7FE38A:

AVATAR BACKGROUND	HEX	CONTRAST WITH THE ICON	WCAG 1.4.11 (3:1)
green	#A8D5BA	1.03:1	 fails
pink	#F6C6D0	1.05:1	 fails
yellow	#F2C879	1.00:1	 fails
blue	#A9C7F0	1.09:1	 fails
lilac	#C9B6E4	1.17:1	 fails
turquoise	#9BD8DE	1.00:1	 fails

Over a photographic tile the same icon measured **9.92:1** — but that number cannot be derived from a hex value, because there the name band is **black at 78 % opacity** and the backdrop is the photograph itself. That figure was therefore measured *on the screen*, not computed from the formula.

WHAT THIS MEANS

A ratio of **1.00** is not “low contrast”. It means the two colours have **mathematically identical luminance** — zero contrast. The person who could not see it was not imagining things: **there genuinely was nothing there**.

2. The formula, precisely

The WCAG 2.1 contrast ratio takes three steps. First, each channel is taken back out of gamma into linear light:

```
s = c / 255
lin = (s <= 0.04045) ? s / 12.92
      : ((s + 0.055) / 1.055) ^ 2.4
```

Then the relative luminance:

```
L = 0.2126·R_lin + 0.7152·G_lin + 0.0722·B_lin
```

And finally the contrast ratio, lighter colour on top:

```
CR = (L_lighter + 0.05) / (L_darker + 0.05)    // 1:1 ... 21:1
```

In programmable form

```
function luminance(hex) {
  const v = [0, 2, 4].map(i => {
    const s = parseInt(hex.substr(i + 1, 2), 16) / 255;
    return s <= 0.04045 ? s / 12.92 : Math.pow((s + 0.055) / 1.055, 2.4);
  });
}
```

```

return 0.2126 * v[0] + 0.7152 * v[1] + 0.0722 * v[2];
}

function contrast(a, b) {
  const la = luminance(a), lb = luminance(b);
  return (Math.max(la, lb) + 0.05) / (Math.min(la, lb) + 0.05);
}

```

The thresholds

WHAT IT COVERS	WCAG CRITERION	AA	AAA
body text	1.4.3 / 1.4.6	4.5:1	7:1
large text (≥24 px, or ≥18.7 px bold)	1.4.3 / 1.4.6	3:1	4.5:1
non-text graphics, UI components, state indicators	1.4.11	3:1	—

The tile icons fall into the **third row**: not text, but the interface cannot be understood without them. The threshold is therefore **3.0** — and the measured 1.00–1.17 is a third of that.

3. What the formula does *not* contain

Look at the three coefficients in the luminance formula:

0.2126 · R	0.7152 · G	0.0722 · B
21 %	72 %	7 %

THE SINGLE MOST IMPORTANT SENTENCE HERE

Hue does not appear in the formula at all. Only lightness does. That is why two colours can sit *opposite* each other on the colour wheel — near-complementary — and still have a contrast ratio of 1.0.

From which follows a rule you can actually program: **never let hue alone distinguish two things. Create a difference in lightness.** Hue may carry extra meaning, but it cannot be the carrier of the meaning — because for a share of your viewers it simply is not there.

The fix did exactly that: the icon did not become a brighter green — it was given a **dark disc with a white glyph**. Hue still speaks (green = message, blue = call), but *visibility* is now carried by the difference in lightness.

4. Live calculator

Contrast between two colours

foreground	hex	background	hex
	#7FE38A		#A8D5BA



L FOREGROUND

0.6129

L BACKGROUND

0.5946

CONTRAST

1.03:1

non-text graphics and UI components (1.4.11 AA, 3:1)

☒ fails

body text (1.4.3 AA, 4.5:1)

☒ fails

body text, enhanced (1.4.6 AAA, 7:1)

☒ fails

The default values are the **actual defective pair**: the original pale green envelope (#7FE38A) on the green avatar's name band (#A8D5BA). To the eye the two colours look entirely different.

5. Colour-vision-deficiency simulation

In practice the **Machado, Oliveira & Fernandes (2009)** matrices are what people use — they are what Chrome DevTools and most design tools run. Three 3×3 matrices, scalable by severity; the full (1.0) cases are used here. The older school, more accurate for dichromats, is **Brettel–Viénot–Mollon (1997/99)**, which projects in LMS space.

⚠ One detail that is often missed: these matrices are defined for **linear RGB**.

Applying them straight to sRGB values gets you close, but it is not correct. On this page the simulation goes sRGB → linear → matrix → sRGB.

The tiles, under simulated vision

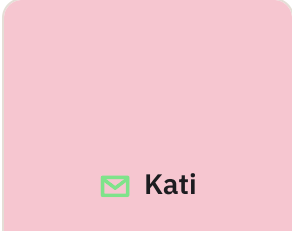
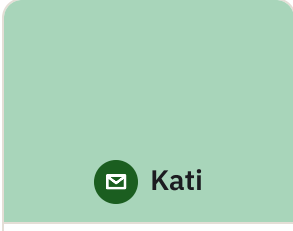
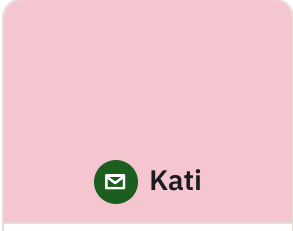
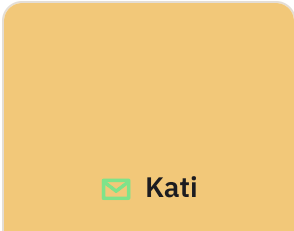
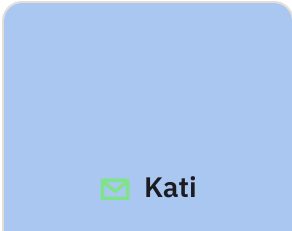
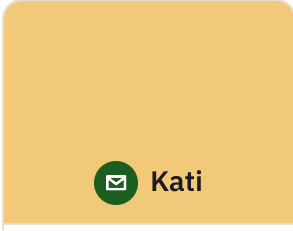
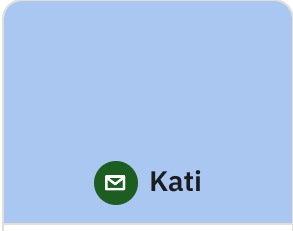
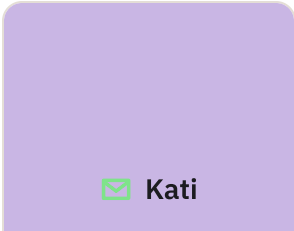
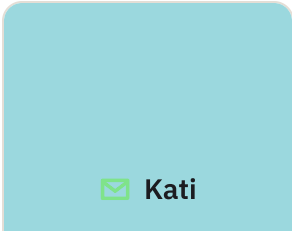
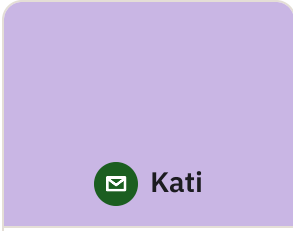
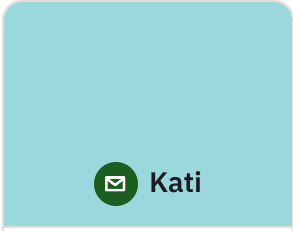
vision mode

normal vision



BEFORE — PALE ICON, NO SURROUND

AFTER — DARK DISC, WHITE GLYPH

 1.03:1 ✗	 1.05:1 ✗	 4.83:1 ✓	 5.21:1 ✓
 1.00:1 ✗	 1.09:1 ✗	 4.99:1 ✓	 4.54:1 ✓
 1.17:1 ✗	 1.00:1 ✗	 4.23:1 ✓	 4.97:1 ✓

WORTH TRYING

Switch to **achromatopsia**. The icons on the left vanish completely — the ones on the right remain exactly as legible as they are in colour. This is the cleanest possible proof that meaning is carried by **lightness**, not by hue.

Notice too that the contrast figures *barely move* between the simulated modes. That is not a bug: these matrices approximately preserve luminance. Which is precisely why **whatever passes on lightness also passes for colour-deficient viewers**.

Testing without writing any code

TOOL	WHERE	WHAT IT GIVES
Chrome DevTools	: → More tools → Rendering → Emulate vision deficiencies	protanopia, deuteranopia, tritanopia, achromatopsia, blurred vision, reduced contrast — <i>on any web page</i>
Android	Developer options → Simulate colour space	deuteranomaly / protanomaly / tritanomaly — <i>on the real app, on a real phone</i>
Color Oracle	colororacle.org	full-screen simulation, Win/Mac/Linux, free
Sim Daltonism	macOS	a movable “lens” you drag around the screen

6. Where WCAG 2.1 gets it wrong

Honesty requires saying it: the formula above is **known to be inaccurate**. It diverges most from what people actually see on dark backgrounds and in the midtones — the $+0.05$ term is a simplification that distorts the dark end.

Its successor is **APCA** (Accessible Perceptual Contrast Algorithm), in the WCAG 3 draft. It models *polarity* as well — dark-on-light and light-on-dark are not the same case — and returns not a ratio but an Lc value roughly between -108 and +106.

THE PRACTICAL DECISION

What is **normative** today — what you can cite on a store listing, in a tender, or at a conference — is **WCAG 2.1**. So that is what we program against. We know APCA, and where it tells us more we listen — but conformance is stated in 2.1.

7. What we do not know

These are our genuine open questions:

1. **Is 3:1 enough on an emergency interface?** WCAG states a minimum, not an optimum. For an emergency button that a frightened, poorly-sighted, elderly person is hunting for — where is the real threshold?
2. **What should we do with the *meaning* of colour, now that lightness carries visibility?** Green = message, blue = call still holds. Does it still help someone who cannot tell the two apart — or does it actively confuse?
3. **Is the severity scale useful in practice?** The Machado matrices scale in steps of 0.1. Is a 0.6 deuteranomaly more realistic than full deuteranopia — or should design always target the worst case?
4. **Is the pastel palette defensible at all?** The six avatar colours are pastel so that they feel friendly and carry dark text. But that is exactly what made a pale icon invisible. Is there a palette that is friendly *and* leaves lightness headroom for anything placed on it?

8. A closing note

This defect was **not found by a machine**. No test of ours failed on it — indeed, in the environment it was designed for (over a dark photograph) the icon was flawless.

A person glanced at the phone and said so. Another, who has difficulty with colour himself, had already reported that he could not find any icon at all — and was told “the little icon is right there”.

He was right, and not subjectively: mathematically. That is what the formula is for — so that nobody has to discover the next one the hard way.

Segítség · vigyzokrad.hu — an emergency-call app and wall console for elderly and convalescing people.

Every number on this page is measured. Contrasts use the WCAG 2.1 relative-luminance formula; the simulation uses Machado–Oliveira–Fernandes (2009) matrices in linear RGB.

An interactive version of this page exists — with the calculator and the simulator live. Ask at the address above.

2026-09-19